IF2211 - Strategi Algoritma

Tugas Besar 1

Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Battlecode 2025

Ramadhian Nabil Firdaus Gumay - 13524126

Faris Wirakusuma - 13524130

Jonathan Harijadi - 13524144

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Bab 1 ..... **2**
Deskripsi Tugas **3**
Latar Belakang 3
Komponen Permainan 3
1. Objectives 3
2. Aturan Tiebreaker 3
3. Peta 3
4. Jarak Pandang dan Pergerakan 4
5. Tower & Ruins 4
6. Pola Sumber Daya Khusus 4
7. Cat 5
8. Chips 5
9. Robot 5
10. Jenis Robot 6
11. Towers 8
● Prosedur Pembangunan Menara 8
● Posisi dan Ukuran Menara 9
12. Serangan 10
13. Penambangan 10
14. Komunikasi 10
Pesan 11
Penanda 12
Bab 2 ..... **15**
Landasan Teori **15**
1. Algoritma Greedy 15
2. Cara Kerja Program Secara Umum 16
Bab 3 ..... **18**
Aplikasi Strategi Greedy **18**
1. Main Bot 18
2. Bot Alternatif 1 18
3. Bot Alternatif 2 18
Penggunaan strategi greedy pada bot ini adalah : 18
Bab 4 ..... **19**
Implementasi dan Pengujian **19**
1. Main Bot 19
2. Bot Alternatif 1 19
3. Bot Alternatif 2 19

Bab 5.....20
Kesimpulan dan Saran..... 20
1. Kesimpulan :..... 20
Lampiran..... 21
Daftar Pustaka.....22

Bab 1
Deskripsi Tugas

Latar Belakang

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi ***greedy*** dalam membuat bot ini.

Komponen Permainan

Komponen-komponen dari permainan ini antara lain:

1. Objectives

   Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.

2. Aturan Tiebreaker

   Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir. Penentuan pemenang dilakukan dengan urutan tiebreaker berikut:

   - Luas area yang sudah diwarnai di peta
   - Jumlah menara sekutu yang masih hidup
   - Total jumlah uang (chips)
   - Jumlah total cat (paint) yang dimiliki semua robot dan menara
   - Jumlah robot sekutu yang masih hidup
   - Jika masih seri, tim dipilih secara acak (uniform random)

3. Peta

   Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara 20×20 hingga 60×60 petak (inklusif). Titik kiri bawah peta memiliki koordinat (0, 0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan

dengan objek MapLocation yang menyimpan nilai x dan y. Agar tidak ada peta yang menguntungkan salah satu pemain, dijamin bahwa peta bersifat simetris (baik melalui rotasi maupun refleksi).

4. Jarak Pandang dan Pergerakan

   Robot selalu bisa melihat fitur peta dan robot lain dalam jarak hingga sqrt(20), kira kira 4.47 petak. Unit tidak bisa bergerak melewati menara atau robot lain. Peta mungkin memiliki tembok (walls), yang tidak bisa dilewati atau diwarnai. Tembok maksimal hanya 20% dari seluruh peta. Tembok tidak bisa dihapus atau dibangun oleh pemain. Robot juga tidak bisa melewati atau mewarnai reruntuhan (ruins).

5. Tower & Ruins

   Menara juga berfungsi sebagai zona spawn tim dan bisa menyerang unit musuh. Untuk membangun menara, tim harus mewarnai pola khusus di atas reruntuhan (ruins), zona 5×5 yang sudah ditentukan di peta *(lihat section towers untuk selengkapnya)*.

   Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan. Pusat reruntuhan berjarak minimal 5 petak satu sama lain, dan zona 5×5 di sekitar pusat reruntuhan dijamin bebas tembok. Setiap tim memulai permainan dengan 2 menara: 1 menara cat dan 1 menara uang. Reruntuhan hanya bisa dilihat oleh robot atau menara jika berada dalam radius penglihatan.

   - Money Tower
   - Paint Tower
   - Defense Tower

   Secara default, setiap tim memulai permainan dengan 1 menara penghasil uang dan 1 menara penghasil cat, keduanya sudah di-upgrade ke level 2.

6. Pola Sumber Daya Khusus

   Mewarnai pola khusus tertentu di peta dapat meningkatkan produksi sumber daya tim. Pola ini disebut **Special Resource Pattern (SRP).** Setiap pola aktif yang berhasil diwarnai akan membuat semua menara tambang sekutu mendapatkan +3 sumber daya tambahan per giliran.

   Robot bisa menandai pola ini menggunakan fungsi **markResourcePattern(),** lalu menyelesaikannya dengan **completeResourcePattern()** (biaya 200 chips). Setelah pola selesai dibuat, pola tersebut harus tetap utuh (tidak rusak warnanya) selama 50 ronde agar menjadi aktif. Pola ini sama untuk semua peta dan permainan.

7. Cat

   Cat adalah sumber daya utama permainan. Digunakan untuk mengklaim wilayah dan menghitung skor. Setiap robot dan menara menyimpan cat secara individu, dengan kapasitas maksimal

tergantung tipenya. Robot yang cat-nya sedikit akan mengalami cooldown lebih lama atau bahkan tidak bisa beraksi. Cat bisa diisi ulang dari mopper atau ditarik dari menara penghasil cat.
Setiap tipe robot memiliki serangan berbeda yang bisa:

- Menggunakan cat untuk meletakkan warna di petak, atau
- Menghapus cat milik lawan dari petak.

Robot bisa memilih antara warna utama atau warna sekunder tim untuk mewarnai petak. Kedua warna ini dianggap sama untuk keperluan skor, tapi dibedakan saat mengecek penyelesaian pola cat.

8. Chips

Chips digunakan untuk membangun unit & menara. Chips dihasilkan setiap ronde oleh menara penghasil uang, dan bisa digunakan oleh robot sekutu mana saja setelah dihasilkan. Setiap tim memulai permainan dengan 2500 chips.

9. Robot

Semua robot dapat melakukan aksi seperti bergerak, merasakan (sensing), dan berkomunikasi satu sama lain. Dalam setiap pertempuran, robot milikmu akan berhadapan dengan satu tim musuh. Permainan berjalan berbasis giliran (turn-based) dan dibagi menjadi ronde. Pada setiap ronde, setiap robot mendapat giliran untuk menjalankan kode dan melakukan aksi. Kode yang dijalankan robot menghabiskan bytecode (ukuran sumber daya komputasi, dijelaskan lebih lanjut pada section bytecode limit). Setiap robot hanya memiliki jumlah bytecode tertentu per giliran; jika habis, gilirannya langsung berakhir dan akan dilanjutkan pada giliran berikutnya. Jika robot sudah selesai menjalankan aksinya pada giliran tersebut, ia harus memanggil **Clock.yield()** untuk menunggu giliran berikutnya dimulai.
Semua robot memiliki HP (hitpoints / nyawa). Ketika HP mencapai nol, robot langsung dihapus dari permainan. Setiap robot memiliki ID unik yang diacak dan tidak kurang dari 10.000. Robot hanya dapat berinteraksi dengan lingkungan terdekat melalui sensing, pergerakan, dan kemampuan khusus. Setiap robot menjalankan salinan kode Anda secara independen. Robot tidak bisa berbagi variabel statis (masing-masing punya salinannya sendiri) karena dijalankan di JVM terpisah.
Dua atau lebih robot tidak boleh berada di petak yang sama. Ketika cooldown pergerakan di bawah 10, robot dapat bergerak ke salah satu dari 8 petak tetangga. Semua robot memiliki

cadangan cat (paint) dengan kapasitas maksimum tertentu. Semakin rendah cadangan cat, semakin tinggi cooldown pergerakan dan aksi yang dihadapi.
Aturan penalti cat rendah:

- Jika cadangan cat hanya X% penuh (X < 50), maka cooldown meningkat (100 – 2X)%.
- Jika cadangan cat habis sama sekali selama giliran, robot tidak bisa bergerak atau melakukan aksi apa pun (kecuali self-destruct) sampai mendapat cat lagi, dan akan kehilangan 20 HP per ronde.

Robot spawn dengan cadangan cat 100% penuh dan dapat diisi ulang di menara atau oleh mopper. Kehilangan cat saat mengakhiri giliran:

- Di wilayah netral : kehilangan 1 cat
- Di wilayah musuh : kehilangan 2 cat
- Di wilayah sekutu : tidak kehilangan cat

Penalti cat tambahan setiap ronde adalah 1 × jumlah robot sekutu yang bersebelahan (dalam 8 petak sekitar). Pinalti ini berlipat ganda (×2) saat berada di wilayah musuh. Robot dapat di-spawn dalam radius aksi √4 (≈ 2 petak) dari menara sekutu, dengan biaya cat dan chips sesuai jenisnya.

10. Jenis Robot

Soldier

- HP: 250
- Kapasitas cat maks: 200
- Biaya produksi: 200 cat + 250 chips
- Serangan: Menyerang satu petak (radius aksi 3)
    - Jika petak kosong atau sudah ada cat sekutu → mewarnai petak tersebut
    - Menimbulkan 50 damage ke menara musuh (tidak ke robot)
- Biaya serangan: 5 cat
- Cooldown aksi: 10

Splasher

- HP: 150
- Kapasitas cat maks: 300

- Biaya produksi: 300 cat + 400 chips
- Serangan: Menyerang area melingkar (radius 2 dari titik pusat)
    - Titik pusat serangan maksimal 2 petak dari posisi splasher
    - Dalam radius √2 (~1.41) dari pusat → bisa menimpa cat musuh
    - Mewarnai semua petak kosong / sekutu di area
    - Menimbulkan 100 damage ke menara musuh (tidak ke robot)
- Biaya serangan: 50 cat
- Cooldown aksi: 50

Mopper

- HP: 50
- Kapasitas cat maks: 100
- Biaya produksi: 100 cat + 300 chips
- Kemampuan utama:
    1. Mop petak (radius √2)
        - Menghapus cat musuh di petak tersebut
        - Jika ada robot musuh di petak → musuh kehilangan 10 cat, mopper mendapat 5 cat
        - Cooldown: 30
    2. Transfer cat ke robot atau menara sekutu (radius √2)
        - Mentransfer jumlah cat tertentu dari cadangan sendiri
        - Cooldown: 10
    3. Ayunan mop (swing) ke salah satu dari 4 arah utama (atas, bawah, kiri, kanan)
        - Menghapus 5 cat dari masing-masing robot musuh yang berdekatan dalam garis lurus
        - Serangan diterapkan dua kali di arah yang sama: → 1 langkah ke arah tersebut → 2 langkah ke arah yang sama
        - Maksimal 6 target potensial (3 di langkah 1 + 3 di langkah 2)
        - Cooldown: 20

Catatan tambahan: Mopper mendapat penalti cat ganda saat bergerak atau mengakhiri giliran di wilayah musuh.

Ability dan Statistik Robot

Name	Health	Paint Capacity	Paint Cost	Money Cost	Attack Cost	Attack Radius	Attack Damage	Attack Cooldown
Soldier	250	200	200	250	5	3	50 health to towers	10
Splasher	150	300	300	400	50	2	100 health to towers	50
Mopper	50	100	100	300	0	$\sqrt{2}$	-10 paint to robots	30

Untuk informasi lebih lengkap, silahkan buka dokumentasi Battlecode pada link berikut.

11. Towers

- Prosedur Pembangunan Menara

  Semua menara dibangun dengan prosedur yang sama. Menara hanya bisa dibangun di atas reruntuhan (ruins), setelah pola cat 5×5 tertentu ditempatkan di sekitar reruntuhan tersebut.

  1. Panggil markTowerPattern() untuk secara otomatis menempatkan penanda di area 5×5 sekitar reruntuhan. Penanda ini menunjukkan posisi mana yang harus diwarnai dengan warna utama (primary color).
     - Biaya: 25 cat
  2. Setelah pola sudah diwarnai, robot bisa memanggil completeTowerPattern() untuk memunculkan menara yang ditentukan di lokasi reruntuhan.
     - Menara akan tetap hidup sampai HP-nya mencapai 0, meskipun pola cat di sekitar reruntuhan kemudian dihapus.
     - Catatan: Pola menara bisa diselesaikan tanpa memanggil markTowerPattern() terlebih dahulu, asalkan pola cat utama dan sekunder sudah cocok.

- Posisi dan Ukuran Menara

  Menara yang dihasilkan akan menempati blok 1×1 di tengah bentuk spawn. Bentuk berbeda menghasilkan jenis menara berbeda dengan rentang serangan, laju serang, laju penambangan, dan kesehatan yang berbeda (lihat tabel di bawah).

  Level dan Upgrade

  - Menara selalu dibangun di level 1.

- Bisa di-upgrade ke level lebih tinggi menggunakan upgradeTower() dalam radius sqrt(2) (~1.41 petak).

Batasan

- Tidak ada tim yang boleh memiliki lebih dari 25 menara sekaligus.
- Setiap menara bisa menyimpan 1000 cat.

Bonus dari Menara Pertahanan

Setiap menara pertahanan yang hidup meningkatkan kekuatan serangan satu blok (single-block attack) dari semua menara sekutu sebesar:

- +5 (level 1)
- +7 (level 2)
- +9 (level 3)

Bonus ini tidak memengaruhi serangan area (AoE) menara sekutu.

Name	Build/ Upgrade cost	Attack Range	Single-block Attack Strength	AoE Attack Strength	Base Mining Rate	Health
Money Lv1	1000 chips	3	20	10	20 chips/turn	1000
Lv2	2500 chips	3	20	10	30 chips/turn	1500
Lv3	5000 chips	3	20	10	40 chips/turn	2000
Paint, Lv1	1000 chips	3	20	10	5 paint/turn	1000
Lv2	2500 chips	3	20	10	10 paint/turn	1500
Lv3	5000 chips	3	20	10	15 paint/turn	2000
Defense, Lv1	1000 chips	4	40	20	20 chips/attack*	2000
Lv2	2500 chips	4	50	25	30 chips/attack*	2500
Lv3	5000 chips	4	60	35	40 chips/attack*	3000

12. Serangan

Menara dapat menyerang unit musuh. Menara memiliki dua jenis serangan:

- Serangan satu petak (single-block attack) yang menyerang satu unit musuh di satu petak.
- Serangan area efek (AoE / area of effect) jarak jauh yang memberikan kerusakan kepada semua unit musuh di dalam jangkauannya.

Setiap giliran, satu menara dapat melakukan satu serangan satu petak dan satu serangan area efek.

13. Penambangan

Setiap giliran, menara penghasil uang dan menara penghasil cat akan secara pasif menambang sumber daya.

- Cat disimpan di dalam menara dan dapat diambil oleh robot sekutu.
- Uang (chips) tersedia untuk semua unit sekutu.

Setiap menara memulai dengan 500 cat untuk digunakan spawn berbagai jenis unit. Menara dapat memunculkan robot dalam jarak 2 petak dari posisinya. Jika cat di menara habis, menara tidak akan bisa spawn unit lagi sampai diisi ulang atau menghasilkan cat sendiri. Cat dapat diisi ulang menggunakan cat yang dikumpulkan oleh mopper.

14. Komunikasi

Robot hanya bisa melihat lingkungan di sekitarnya secara langsung dan dikendalikan secara independen oleh salinan kode yang sama, sehingga koordinasi antar-robot sangat sulit. Anda tidak dapat berbagi variabel apa pun di antara mereka; bahkan variabel static pun tidak akan dibagikan karena setiap robot memiliki salinannya sendiri.

Komunikasi dilakukan dengan dua cara:

- Pesan (Messages) Pesan dapat berisi informasi apa saja, tetapi dibatasi hanya 4 byte.
- Penanda (Markers) Penanda menyatakan warna apa yang seharusnya digunakan untuk mewarnai ulang suatu petak, dan penanda ini terlihat oleh semua robot sekutu. Penanda tidak memberikan efek lain apa pun dalam permainan.

Pesan

Robot atau menara yang berada dalam jangkauan satu sama lain dapat saling mengirim pesan. Sebuah robot dianggap “dalam jangkauan” (in range) sebuah menara jika memenuhi dua syarat sekaligus:

- Jaraknya ke menara tersebut ≤ sqrt(20) (~4.47 petak), dan
- Robot tersebut terhubung dengan menara melalui jalur cat sekutu (artinya robot bisa bergerak sampai ke menara hanya dengan berjalan di petak-petak yang sudah memiliki cat milik timnya sendiri).

The robots and towers who are connected by paint can talk, but the robot who is not on paint cannot talk to the tower.

Setiap robot dan menara menyimpan semua pesan yang diterima dalam 5 ronde terakhir di dalam buffer-nya. Setelah 5 ronde berlalu, pesan tersebut akan dihapus secara otomatis.

Batas Pengiriman Pesan per Ronde adalah sebagai berikut

- Robot: maksimal 1 pesan per ronde
- Menara: maksimal 20 pesan per ronde

Setiap pesan terdiri dari:

- Satu bilangan bulat 32-bit (integer)
- Nomor ronde saat pesan dikirim
- ID robot yang mengirim pesan

Menara memiliki kemampuan tambahan untuk melakukan broadcast (siaran) pesan ke menara sekutu lain yang berada dalam jarak sqrt(80) (~8.94 petak).

- Menara tidak perlu terhubung melalui jalur cat untuk melakukan broadcast ini.
- Setiap kali broadcast dilakukan, itu dihitung sebagai satu pesan terhadap batas 20 pesan per ronde menara.
- Kemampuan ini memungkinkan menara memperluas jangkauan komunikasi jarak jauh antar-menara.

Penanda

Penanda adalah informasi yang bisa ditempatkan atau dihapus di peta oleh robot. Penanda hanya memiliki dua jenis:

- Menandai dengan warna sekunder
- Menandai dengan warna utama

Penanda hanya terlihat oleh robot sekutu dan akan tetap ada di petak tersebut sampai dihapus atau diganti dengan penanda lain oleh sekutu. Selain itu, penanda hanya dapat ditempatkan pada ubin yang dapat dicat

Penanda dapat diletakkan dengan cara

- Secara manual: robot dapat menempatkan penanda di petak yang berjarak ≤ sqrt(2) (~1.41 petak) dari posisi robot saat ini. Akan dijelaskan lebih lanjut pada section Action
- Secara otomatis melalui fungsi pola:
    - markTowerPattern() → menandai pola 5×5 di sekitar reruntuhan dengan penanda yang sesuai untuk membangun menara
    - markResourcePattern() → menandai pola 5×5 untuk Special Resource Pattern (SRP)

15. Action and Cooldown

Robot melakukan berbagai aksi untuk berinteraksi dengan dunia permainan. Beberapa aksi tidak bisa dilakukan berkali-kali dalam satu giliran atau dalam waktu singkat karena terikat cooldown. Berikut daftar aksi utama:

- Menyerang/mewarnai

  Menyerang dan mewarnai dilakukan dengan fungsi yang sama. Aksi ini hanya bisa dilakukan jika action cooldown robot < 10. Action cooldown robot dapat dicek dengan fungsi isActionReady(). Dilakukan dengan memanggil metode attack() yang akan menaargetkan satu petak yang dapat dilihat oleh robot tersebut. Efeknya tergantung tipe robot (seperti yang dijelaskan di bagian Robot): memberikan damage atau mewarnai petak. Setelah dilakukan, action cooldown robot bertambah sesuai nilai yang ditentukan oleh tipe robot.
- Bergerak (moving)

  Hanya bisa dilakukan jika movement cooldown < 10. Movement cooldown dapat dicek dengan isMovementReady(). Petak tujuan harus kosong (tidak ditempati unit lain) dan bisa dilewati (passable). Dilakukan dengan memanggil move() dengan arah yang

diinginkan. Setelah bergerak, movement cooldown bertambah 10. Untuk memeriksa apakah gerakan valid, gunakan canMove().

- Menandai (marking)

  Semua robot bisa menandai petak tempat robot sedang berdiri, atau Menghapus penanda sekutu dari petak tempat robot berdiri. Biaya dari aksi ini adalah 1 cat. Aksi ini tidak menambah action cooldown sama sekali. Aksi dapat dilakukan dengan mark() atau removeMark(). Penanda tidak memengaruhi kepemilikan wilayah (tidak membuat petak menjadi wilayah sekutu). Penanda hanya bisa dirasakan oleh robot sekutu melalui senseNearbyMapInfos(). Penanda secara umum tersebut hanya boleh ditempatkan pada petak yang dapat diwarnai (tidak ada tembok atau reruntuhan). Selain itu, terdapat Fungsi khusus penanda, yakni markTowerPattern() dan markResourcePattern()menggunakan 25 cat dan menandai area 5×5 dengan pola yang sesuai.

- Mentransfer Cat (Transferring)

  Hanya dapat dilakukan oleh robot mopper, dan hanya dapat dilakukan jika action cooldown < 10 (dapat dicek dengan isActionReady()). Aksi ini dapat dilakukan dengan memanggil prosedur transferPaint(). Jarak maksimal: √2 (~1.41 petak). Mentransfer jumlah cat tertentu ke target (robot atau menara sekutu). Jika jumlah yang diberikan negatif, maka mopper justru mengambil cat dari target. Setelah dilakukan, action cooldown bertambah sesuai nilai yang disebutkan di deskripsi mopper.

- Menarik cat dari menara (withdrawing)

  Semua robot bisa menarik cat dari menara sekutu untuk mengisi ulang cadangan cat mereka. Memerlukan action cooldown < 10. Setelah dilakukan, action cooldown bertambah 10. Jarak maksimal: √2 (~1.41 petak). Dilakukan dengan transferPaint() menggunakan nilai negatif untuk menunjukkan jumlah yang ingin ditarik.

- Spawn Robot (spawning)

  Hanya bisa dilakukan jika action cooldown menara < 10 (dicek dengan isActionReady()). Dilakukan dengan buildRobot(). Mengurangi biaya robot (cat + chips) dari stok menara. Memunculkan satu robot tipe tertentu di petak yang ditentukan. Setelah dilakukan, action cooldown menara bertambah 10.

- Berkomunikasi (communitating)

  Robot menggunakan sendMessage() untuk mengirim pesan ke menara sekutu dalam jangkauan. Menara menggunakan sendMessage() untuk mengirim pesan ke robot sekutu dalam jangkauan. Robot menggunakan readMessages() untuk membaca pesan baru yang diterima dalam 5 ronde terakhir dari buffer. Kedua metode ini tidak menambah action

cooldown. Syarat komunikasi: kedua unit harus berada di petak yang terhubung melalui jalur cat sekutu.

- Menghancurkan Diri Sendiri (Disintegrating)

  Robot apa pun bisa memanggil disintegrate(). Menghancurkan robot tersebut secara instan. Tidak ada cooldown khusus untuk aksi ini.

Bab 2

Landasan Teori

1. Algoritma Greedy

   Algoritma Greedy merupakan salah satu paradigma utama dalam desain algoritma yang berfokus pada penyelesaian persoalan optimasi. Prinsip fundamental dari algoritma ini adalah mengambil serangkaian keputusan secara bertahap untuk membangun sebuah solusi. Pada setiap langkah, algoritma greedy akan selalu membuat pilihan yang tampak paling menguntungkan atau paling optimal pada saat itu juga, tanpa mempertimbangkan konsekuensi dari pilihan tersebut di masa depan.

   Pendekatan ini didasarkan pada harapan heuristik bahwa dengan mengumpulkan serangkaian pilihan *local optimum*, algoritma pada akhirnya akan mencapai solusi optimal secara keseluruhan (*global optimum*).

   Algoritma greedy memiliki enam komponen dasar penyusun solusi, yaitu:

   a. Himpunan Kandidat, yaitu elemen-elemen dasar atau opsi aksi yang tersedia dan dapat dipertimbangkan untuk dimasukkan ke dalam solusi pada setiap langkah.
   b. Himpunan Solusi, yaitu kumpulan elemen atau aksi yang telah dipilih dari himpunan kandidat dan dieksekusi untuk membentuk solusi akhir.
   c. Fungsi Solusi, yaitu sebuah fungsi boolean yang menentukan apakah himpunan solusi yang telah terbentuk pada iterasi saat ini sudah merupakan solusi yang lengkap dan final untuk menghentikan algoritma pencarian.
   d. Fungsi Kelayakan, yaitu sebuah fungsi validasi yang bertugas memeriksa apakah sebuah kandidat dapat dimasukkan ke dalam himpunan solusi tanpa melanggar batasan-batasan dari permasalahan yang ada.
   e. Fungsi Seleksi, yaitu fungsi utama yang mengevaluasi nilai dari masing-masing kandidat yang layak. Fungsi ini yang menentukan kandidat mana yang paling memberikan keuntungan sesaat tertinggi untuk dipilih.
   f. Fungsi Objektif, yaitu fungsi metrik yang mengukur nilai atau kualitas dari solusi untuk menentukan apakah tujuan akhir dari permasalahan telah tercapai.

   Karakteristik utama dari algoritma greedy adalah keputusan yang telah diambil tidak dapat dievaluasi ulang atau dibatalkan. Hal ini membuat algoritma greedy memiliki kompleksitas waktu yang sangat efisien dalam komputasi, meskipun kelemahannya adalah tidak selalu menjamin penemuan *global optimum* pada beberapa kasus permasalahan tertentu.

2. Cara Kerja Program Secara Umum

Program ini merupakan implementasi dari bot di dalam lingkungan *real-time strategy* yang berjalan dengan sistem berbasis giliran. Mekanisme arsitektur pada program ini adalah sebagai berikut:

a. Cara Bot Melakukan Aksinya

Dalam arsitektur *engine* Battlecode, setiap robot menjalankan salinan kode secara independen di dalam *Java Virtual Machine* (JVM) yang terpisah, sehingga tidak ada pembagian memori atau variabel statis antar agen. Siklus hidup aksi bot diatur oleh beberapa batasan sistem:

i. Batasan Komputaasi (Bytecode): Setiap agen memiliki jatah eksekusi intruksi komputasi maksimal per giliran (17.500 *bytecode* untuk robot dan 20.000 untuk menara). Jika komputasi mencapai batas ini di tengah proses, status agen akan dibekukan dan dilanjutkan secara otomatis pada ronde berikutnya. Untuk mencegah inkonsistensi *state* akibat interupsi komputasi ini, agen menggunakan perintah Clock.yield() di akhir algoritmanya untuk mengakhiri gilirannya secara eksplisit.

ii. Manajemen State dan Sensor: Agen hanya dapat berinteraksi dengan dunia luar melalui metode *sensing* dengan jarak pandang maksimal ≈ 4.47 petak (radius $\sqrt{20}$). Agen memindai lingkungan untuk mengenali entitas kawan, lawan, ubin, dan reruntuhan.

iii. Manajemen Sumber Daya dan Cooldown: Aksi dunia fisik seperti bergerak, menyerang/mewarnai, membangung menara, atau mentransfer cat membutuhkan sumber daya (Cat dan *Chips*). Selain itu, aksi dibatasi oleh waktu tunggu yang dipengaruhi oleh sisa kapasitas cat dari robot tertentu.

b. Cara Mengimplementasikan Algoritma Greedy ke Dalam Bot

Implementasi algoritma *greedy* bertindak sebagai otak dari bot setiap siklus gilirannya. Pemetaannya adalah sebagai berikut:

i. Setiap ronde, agen memindai lingkungan untuk mengumpulkan himpunan kandidat, seperti daftar ubin di sekitarnya, daftar musuh, atau arah pergerakan yang memungkinkan.

ii. Agen menjalankan fungsi kelayakan dengan memeriksa status internalnya, seperti ketersediaan cat, kondisi isActionReady(), kondisi isMovementReady(), serta apakah target ubin kosong. Kandidat yang melanggar batas akan dibuang.

iii. Terhadap kandidat yang layak, agen menerapkan fungsi seleksi. Bot akan memberikan skor evaluasi pada setiap kandidat berdasarkan metrik tertentu. Misalnya, secara *greedy* memilih target serangan yang paling banyak menghapus cat musuh, atau memilih pergerakan menuju reruntuhan dengan jarak absolut terpendek.

iv. Aksi dieksekusi berdasarkan nilai heuristik tertinggi/terendah secara instan demi berkontribusi pada fungsi objektif utama permainan, yaitu mendominasi minimal 70% ubin peta dengan cat sekutu atau menghancurkan seluruh entitas lawan.

c. Cara Menjalankan Bot:

Secara teknis, lingkungan pengembangan dan eksekusi bot dikelola menggunakan sistem *build automation* Gradle. Langkah-langkah untuk menjalankan bot pada simulasi lokal adalah sebagai berikut:

i. Melakukan duplikasi repositori sistem ke penyimpanan lokal menggunakan perintah git clone https://github.com/Fariz36/STIMA-battle.

ii. Melakukan proses kompilasi kode *source* bot dengan menjalankan perintah ./gradlew build di terminal pada direktori proyek.

iii. Mengakses direktori antarmuka pengguna (cd client) dan menjalankan aplikasi client bawaan.

iv. Pada menu konfigurasi aplikasi *client*, mengatur direktori permainan sebagai *directory root* agar engine dapat membaca hasil kompilasi dari agen bot yang telah dibuat.

Bab 3

Aplikasi Strategi Greedy

1. Main Bot

   Greedy Algorithm yang digunakan di dalam bot ini adalah:

   - Pada Tower: Utamakan upgrade jika bisa (priority upgrade> spawn bot) tetapi dengan syarat uang yang terdapat pada team telah memenuhi batas greatResourceForUpgrade namun jika tidak memenuhi maka langkahnya langsung spawn robot jika bisa melakukannya atau score mencukupi dengan kata lain jika kita memiliki resource yang sangat banyak maka kemungkinan pattern nya (upgrade,upgrade,..,spawn algoritmbest1,spawn algoritmbest1,spawn algoritmbest2) ,lalu bagaimana mendapatkan spawn algoritm best nya?,cara nya adalah membuat system scoring secara dinamis (tergantung dari kondisi tower) secara lokal,dengan spesifikasi sebagai berikut:
     a) Jika pada di move n/local move n tower : enemy detected>0 maka prioritaskan mopper
     b) Jika pada di move n/local move n tower : chips>1500 dan paint > `splasherPaintReq` (requirement round terbaik untuk painting) atau chips dan paint cost memenuhi untuk spawn Splasher maka akan langsung spawn splasher jadi bisa dibilang splasher ini prioritas dalam default (dalam kondisi tertentu) sehingga akan langsung spawn ketika tower upgrade tidak memenihi di awal awal dan juga dapat di spawn
     c) Jika didalam keadaan selain requirement spalsher dan mopper dan juga jumlah soldier dalam kondisi ideal maka akan menspawn soldier
   - Pada Robots:

     Setiap robot lokal akan bergerak jika `cooldown < 10`. Namun, karena Battlecode menyediakan fungsi `isActionReady()` dan `isMovementReady()`, pengecekan *cooldown* secara manual tidak lagi diperlukan. Bot menggunakan kedua *boolean* tersebut untuk menjalankan strategi priority greedy (Movement > Action).

     Baik pergerakan maupun aksi memiliki algoritma *greedy* masing-masing yang berbasis pada local_radius_scoring, di mana keputusan diambil berdasarkan skor tertinggi di dalam radius sensor. Untuk menentukan skor aksi terbaik (*ActionPower*), bot menggunakan bobot inisialisasi (`initializeWeights()`) yang disesuaikan dengan jenis unit: Mopper diprioritaskan untuk transfer sumber daya, sedangkan Splasher dan Soldier difokuskan pada *damage* ke tower lawan.ActionPower dapat dihitung dengan:

     $$ActionPower = \Delta paintval + \Delta attackval + \Delta supportval$$

     Nilai ActionPower dihitung melalui akumulasi dari paintval, attackval, dan supportval. Variabel paintval ditentukan berdasarkan hasil penjumlahan antara bobot paint empty dikali jumlah tile kosong serta bobot paint enemy dikali jumlah tile lawan yang terdeteksi di seluruh arah sekitar robot. Sementara itu, attackval diperoleh dari deteksi keberadaan

musuh di seluruh arah dalam jangkauan serangan, dan supportval dihitung berdasarkan jumlah unit kawan yang berada dalam radius robot.

Untuk mekanisme pergerakan, keputusan ditentukan oleh variabel weightTowerAttack dan weightDistanceImprove yang dipengaruhi oleh nilai multiplier tertentu. Nilai multiplier ini dihasilkan dari pemindaian menara musuh serta ketersediaan cadangan cat pada robot. Jika menara musuh terdeteksi dan cadangan cat mencukupi, maka multiplier akan bernilai tinggi untuk memicu serangan. Namun, jika cadangan cat kosong, robot tetap akan melakukan serangan dengan metode bunuh diri ke arah musuh guna memastikan unit tidak terbuang sia-sia tanpa memberikan dampak. Apabila syarat tersebut tidak terpenuhi, maka multiplier tidak akan memberikan pengaruh signifikan terhadap pergerakan. Dalam pencarian weightDistanceImprove, algoritma akan memprioritaskan pengurangan jarak terhadap target strategis berdasarkan bobot yang telah ditetapkan.

Strategi spesifik untuk setiap jenis robot diterapkan untuk memaksimalkan peran mereka di lapangan. Soldier difokuskan sebagai unit penyerang utama dengan memprioritaskan skor pada musuh yang memiliki kesehatan rendah untuk memastikan eliminasi cepat. Splasher menggunakan mekanisme area of effect dengan memprioritaskan ubin yang dikelilingi oleh banyak ubin lawan agar efisiensi penggunaan cat meningkat secara signifikan. Mopper memiliki logika khusus untuk mendeteksi ubin yang terkena cat lawan dalam jumlah besar guna dibersihkan, serta memprioritaskan transfer cat ke unit kawan yang kehabisan sumber daya. Sementara itu, unit pendukung lainnya akan bergerak mengikuti formasi kawan untuk memaksimalkan nilai supportval dalam perhitungan strategi lokal.

2. Bot Alternatif 1

Penggunaan strategi greedy pada bot ini adalah :

- Pada Tower: Pada fase awal tower, tower akan memprioritaskan memabngun soldier dan menghasilkan resource. Tower akan mensummon mopper dan splasher di sekitaran midgame hingga endgame. Tower yang dibuat akan bergantian antara money tower yang menghasilkan chip dan paint tower yang menghasilkan cat. Tower dapat menentukan apakah sudah melewati midgame atau belum melalui jumlah turnnya. Jika turnnya sudah melewati 200 maka dinyatakan midgame dan tower mulai mensummon mopper dan splasher.
- Pada Robots: Strategi greedy yang digunakan soldier adalah pertama soldier akan mencari ruin terdekat untuk membuat tower, setelah membuat tower, soldier akan mencoba untuk mengupgrade tower tersebut. Prioritas utama soldier adalah membuild tower dan mengupgrade tower. Soldier juga akan mewarnai tile yang sedang dijalani. Soldier disini berguna untuk melakukan greedy expansion di awal permainan. Jenis tower yang dibangun akan bergantian antara Paint Tower dan Money Tower. Pola ini dibuat untuk menjaga keseimbangan antara sumber daya paint dan chips yang dibutuhkan selama permainan.

Jika tidak ditemukan ruin di sekitar robot, maka robot akan mencari tower sekutu terdekat. Apabila tower tersebut dapat diupgrade, maka robot akan melakukan upgrade pada tower tersebut. Jika tidak terdapat tower yang dapat di-upgrade, robot akan bergerak secara acak untuk melakukan eksplorasi peta.

Unit Mopper berfungsi sebagai unit pembersih paint musuh pada peta. Strategi greedy yang digunakan adalah memprioritaskan tile yang memiliki enemy paint. Berikut prioritas dari unit mopper:

1) Jika tile dapat diserang dengan mopSwing → gunakan mopSwing
2) Jika tile dapat diserang dengan attack → lakukan attack
3) Jika tidak dapat menyerang → bergerak menuju tile tersebut

Unit Splasher menggunakan strategi greedy berbasis perhitungan skor tile untuk menentukan target serangan. Perhitungan skor yg dilakukan dengan cara menambah 5 untuk setiap tile musuh dalam area serangan, menambah 3 untuk menyerang daerah netral, dan mengurangi skornya sesuai dengan jarak target ke robot. Jika skor tinggi dan target tidak berada di area serangan maka splasher akan mendekati target tersebut. Hal ini memungkinkan splasher menyerang dengan efektif selagi menunggu cooldown.

3. Bot Alternatif 2

Algoritma greedy yang digunakan bot ini adalah:

- Pada Tower: Tower menggunakan prioritas spawn robot berdasarkan kondisi resource dan kondisi tim di sekitar tower. Sistem yang digunakan bersifat greedy karena tower akan selalu memilih aksi yang paling menguntungkan pada kondisi saat itu (local optimum) tanpa mempertimbangkan strategi jangka panjang. Urutan prioritas tower dalam mengambil keputusan adalah sebagai berikut:
    1) Menyerang musuh terlebih dahulu jika ada musuh dalam jangkauan : Jika tower mendeteksi robot musuh dalam radius serang dan dapat melakukan attack, maka tower akan langsung menyerang target tersebut sebelum melakukan aksi lain. Hal ini bertujuan untuk mempertahankan area sekitar tower dari ancaman musuh.
    2) Melakukan spawn robot berdasarkan kondisi tim di sekitar tower : Tower akan memeriksa jumlah robot ally di sekitar yang memiliki paint rendah (di bawah 30% kapasitas). Jika jumlah robot yang kekurangan paint cukup banyak, maka tower akan memprioritaskan spawn Mopper untuk membantu membersihkan area serta mendistribusikan paint ke unit lain.
    3) Jika tidak ada kebutuhan mendesak untuk Mopper, maka tower akan melihat kondisi round permainan : Pada fase awal permainan (round < 80), tower akan memprioritaskan spawn Soldier karena unit ini memiliki fungsi utama untuk melakukan eksplorasi map serta membangun tower baru dari ruin yang ditemukan.
    4) Jika kondisi early game sudah terlewati dan resource mencukupi, tower akan memprioritaskan spawn Splasher : Splasher diprioritaskan karena kemampuannya melakukan area painting, yang dapat memperluas kontrol wilayah secara lebih efisien dibanding unit lainnya.

5) Jika semua kondisi di atas tidak terpenuhi, maka tower akan kembali melakukan spawn Soldier sebagai unit default untuk menjaga jumlah unit tetap stabil di dalam tim.

- Pada Robots : Setiap robot akan mengecek apakah bisa melakukan action atau movement dan bot ini memprioritaskan movement ketimbang action. Robot akan mencoba bergerak menuju target atau area strategis terlebih dahulu, kemudian melakukan aksi seperti attack, paint, atau support. Robot akan menghitung sesuai variabel lokal untuk memilih target terbaik sesaui dengan deteksi sensor. Secara umum skor aksi dapat dianggap sebagai kombinasi dari beberapa komponen berikut:

  $$ActionScore = paintScore + attackScore + supportScore$$

  paintScore diperoleh dari jumlah tile yang dapat dicat atau dibersihkan di sekitar robot. Tile musuh akan memiliki prioritas lebih tinggi dibanding tile kosong. Sedangkan attackScore diperoleh dari keberadaan musuh atau target strategis yang dapat diserang oleh robot. Terakhir, supportScore diperoleh dari keberadaan unit ally di sekitar robot yang membutuhkan bantuan seperti transfer paint. Dari score ini, robot akan melakukan action dengan score terbesar.

  Pergerakan robot dilakukan dengan mekanisme berikut:

  1) Jika robot memiliki target khusus (misalnya ruin atau tower), robot akan bergerak langsung menuju target tersebut.
  2) Jika tidak ada target spesifik, robot akan bergerak menuju target eksplorasi.
  3) Jika robot mengalami kondisi stuck selama beberapa turn, arah pergerakan akan diubah secara random untuk menghindari deadlock.

Bab 4

Implementasi dan Pengujian

1. Main Bot

   a) Implementasi

Implementasi Pseudocode Pada Decision Robot (strategi greedy pada robot)

```
// Logic utama pada Robot
function finalDecision():
   updateDynamicStrategy() // Update bobot jika cat habis (greedy mode: cari tower)

   // Greedy Action (ActionPower)
   if isActionReady() then
      bestTarget ← null
      maxScore ← -infinity

      // Cek prioritas tertinggi: Perbaikan Ruin/Pola Tower
      if findRuinInRange() then
         executeRuinPattern()
         return

      // Cari musuh/kawan dengan skor ActionPower tertinggi
      targets ← senseNearbyRobots(actionRadius)
      for each target in targets:
         score ← ActionPower(bobot_berdasarkan_tipe_unit)
         if score > maxScore then
            maxScore ← score
            bestTarget ← target

      // Jika tidak ada unit, cek ubin lantai (paintval)
      if bestScore < 1 then
         for each tile in directions:
            score ← ActionPower(info_ubin, paint_empty, paint_enemy)
            if score > maxScore then
               maxScore ← score
               bestTarget ← tile

      executeAction(bestTarget)

   // Greedy Movement (PositionPower)
   if isMovementReady() then
      bestDir ← CENTER
      maxScore ← -infinity

      for each dir in directions:
         if canMove(dir) then
```

```
            // dist_imp: seberapa jauh jarak berkurang ke target (greedy approach)
            score ← PositionPower(tower_opp, dist_imp, ally_paint_penalty)
            if score > maxScore then
               maxScore ← score
               bestDir ← dir

      move(bestDir)
```

Implementasi Pseudocode Pada Decision Tower (strategi greedy pada towers)

```
// Logic utama pada Tower
function finalDecision():
   if not isActionReady() then return

   // 1. Greedy Attack: Cari HP terendah
   enemies ← senseNearbyEnemies()
   if enemies.length > 0 then
      target ← findEnemyWithMinimumHealth(enemies)
      if canAttack(target) then
         attack(target)

   // 2. Greedy Upgrade vs Spawn
   // Utamakan upgrade jika sumber daya melimpah
   if canUpgrade() and chips > greatResourceForUpgrade then
      upgradeTower()
   else:
      executeGreedySpawn()

function executeGreedySpawn():
   // Tentukan tipe berdasarkan komposisi tim saat ini
   if enemyIsMopper then toBuild ← SOLDIER
   else
      if noMoppers then toBuild ← MOPPER
      else
         if chips_high and paint_high then toBuild ← SPLASHER
         else toBuild ← SOLDIER (ideal count)

   // Cari arah spawn yang tersedia (rotasi greedy)
   spawnDirection ← directionTo(nearestEnemy)
   for 8 attempts:
      if canBuild(toBuild, spawnDirection) then
         build(toBuild)
         return
      spawnDirection ← rotateLeft(spawnDirection)
```

b) Pengujian

Proses pengujian bot dilakukan melalui terminal dengan mengeksekusi perintah:

./gradlew run -PteamA=main_bot -PteamB=alt_bot_1-Pmaps=DefaultSmall

./gradlew run -PteamA=main_bot -PteamB=alt_bot_2-Pmaps=DefaultSmall

Perintah tersebut akan menjalankan simulasi pertandingan antara main_bot melawan alt_bot_1 dan alt_bot_2 pada peta DefaultSmall, di mana main_bot berperan sebagai Team A. Pengujian dengan skema ini memungkinkan kamu untuk melihat apakah main_bot memiliki performa yang konsisten saat menghadapi berbagai tipe lawan yang ditempatkan pada alt_bot.

c) Hasil:

```
[A: #12723@1802] I'm alive
[A: #11854@1813] I'm alive
[A: #11666@1816] I'm alive
[A: #12102@1826] I'm alive
[A: #13489@1834] I'm alive
[A: #12883@1837] I'm alive
[A: #14085@1846] I'm alive
[A: #12497@1853] I'm alive
[A: #10056@1859] I'm alive
[A: #13678@1866] I'm alive
[A: #13454@1873] I'm alive
[A: #13065@1880] I'm alive
[A: #13910@1886] I'm alive
[A: #13692@1894] I'm alive
[A: #13625@1906] I'm alive
[A: #10365@1913] I'm alive
[A: #13301@1915] I'm alive
[A: #12786@1926] I'm alive
[A: #12709@1929] I'm alive
[A: #11424@1936] I'm alive
[A: #10630@1942] I'm alive
[A: #10192@1950] I'm alive
[A: #10478@1957] I'm alive
[A: #12118@1964] I'm alive
[A: #13690@1973] I'm alive
[A: #13319@1977] I'm alive
[A: #13115@1986] I'm alive
[A: #13442@1993] I'm alive
[A: #13016@2000] I'm alive
[server]                    main_bot (A) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------

BUILD SUCCESSFUL in 3s
8 actionable tasks: 6 executed, 2 up-to-date
```

```
t -PteamB=alt bot 2 -Pmaps=DefaultSmall
[A: #14085@1804] I'm alive
[A: #12497@1811] I'm alive
[A: #10056@1818] I'm alive
[A: #13678@1825] I'm alive
[A: #13454@1833] I'm alive
[A: #13065@1839] I'm alive
[A: #13910@1846] I'm alive
[A: #13692@1854] I'm alive
[A: #13625@1860] I'm alive
[A: #10365@1868] I'm alive
[A: #13301@1874] I'm alive
[A: #12786@1881] I'm alive
[A: #12709@1888] I'm alive
[A: #11424@1896] I'm alive
[A: #10630@1901] I'm alive
[A: #10192@1910] I'm alive
[A: #10478@1918] I'm alive
[A: #12118@1923] I'm alive
[A: #13690@1930] I'm alive
[A: #13319@1936] I'm alive
[A: #13115@1944] I'm alive
[A: #13442@1950] I'm alive
[A: #13016@1958] I'm alive
[A: #11306@1964] I'm alive
[A: #13424@1975] I'm alive
[A: #11002@1981] I'm alive
[A: #13188@1985] I'm alive
[A: #13328@1996] I'm alive
[A: #12852@1999] I'm alive
[server]                    main_bot (A) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------

BUILD SUCCESSFUL in 2s
8 actionable tasks: 3 executed, 5 up-to-date
```

2. Bot Alternatif 1

   a) Implementasi

Implementasi Pseudocode Pada Splasher

```
// Inisialisasi variabel
MapLocation best ← null
integer bestScore ← -999

//
for (MapInfo tile : tiles)
   integer score ← 0

   if (tile.getPaint().isEnemy()) then score ← score +5
   if (tile.getPaint() = PaintType.EMPTY) then score ← score + 3

   integer dist ← rc.getLocation().distanceSquaredTo(tile.getMapLocation())
```

```
score ← score - dist

if (score > bestScore) then
    bestScore ← score
    best ← tile.getMapLocation()
```

Implementasi Pseudocode Pada Soldier memilih ruin terdekat

```
MapInfo ruin ← null;
int bestDist ← 999999;

for (MapInfo tile : nearby)
    if (tile.hasRuin()) then
        integer dist ← rc.getLocation().distanceSquaredTo(tile.getMapLocation());
        if (dist < bestDist) then
            bestDist ← dist;
            ruin ← tile;
```

b) Pengujian

Proses pengujian bot dilakukan melalui terminal dengan mengeksekusi perintah:

./gradlew run -PteamA=alt_bot_1 -PteamB=main_bot-Pmaps=DefaultSmall

./gradlew run -PteamA=alt_bot_1 -PteamB=alt_bot_2-Pmaps=DefaultSmall

Perintah tersebut akan menjalankan simulasi pertandingan antara alt_bos_1 melawan main_bot dan alt_bot_2 pada peta DefaultSmall, di mana main_bot berperan sebagai Team A. Pengujian dengan skema ini memungkinkan kamu untuk melihat apakah main_bot memiliki performa yang konsisten saat menghadapi berbagai tipe lawan yang ditempatkan pada alt_bot.

c) Hasil

```
_2 -PteamB=alt_bot_1 -Pmaps=DefaultSmall

> Task :run
[server] -------------------- Match Starting --------------------
[server] alt_bot_2 vs. alt_bot_1 on DefaultSmall
[server]                    alt_bot_1 (B) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------

BUILD SUCCESSFUL in 3s
8 actionable tasks: 3 executed, 5 up-to-date
```

```
[B: #10466@1940] I'm alive
[B: #13342@1951] I'm alive
[B: #11710@1960] I'm alive
[B: #10449@1971] I'm alive
[B: #11483@1980] I'm alive
[B: #12526@1991] I'm alive
[B: #10282@2000] I'm alive
[server]                    main_bot (B) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------

BUILD SUCCESSFUL in 3s
8 actionable tasks: 3 executed, 5 up-to-date
```

3. Bot Alternatif 2
    a) implementasi

Strategi Greedy Soldier:

```
function finalDecisionSoldier():
  updateDynamicStrategy() // Jika cat < 20%, prioritaskan cari Tower terdekat

  // Greedy Action
  if isActionReady() then
    // Prioritas 1: Perbaikan Ruin/Pola Tower (Greedy Ruin)
    if findRuinInRange() then
      markTowerPattern()
      targetTile ← findBestPatternTile() // Cari ubin yang butuh cat sesuai pola
      if targetTile != null then
        executeAttack(targetTile)
        return

    // Prioritas 2: Cari ubin musuh/kosong (Paint Value)
    bestTarget ← null
    maxScore ← -infinity

    for each tile in nearbyTiles:
      if canAttack(tile) and not tile.isAlly() then
        // ActionPower: Musuh (3), Kosong (1), dikurangi penalty jarak
        score ← (tile.isEnemy() ? 3  : 1) - distanceTo(tile)
        if score > maxScore:
          maxScore ← score
          bestTarget ← tile

    if bestTarget != null then
```

```
        executeAttack(bestTarget)

    // Greedy Movement
    if isMovementReady() then
        bestDir ← CENTER
        maxScore ← -infinity

        // Jika ada Ruin, dist_imp fokus ke Ruin, jika tidak ke targetLoc/Wander
        target ← (closestRuin != null) ? closestRuin : targetLoc

        for each dir in directions:
            if canMove(dir) then
                // PositionPower: Fokus pada pengurangan jarak ke target (Greedy Move)
                score ← -distanceSquared(move(dir), target)
                if score > maxScore then
                    maxScore ← score
                    bestDir ← dir
        move(bestDir)
```

Strategi Greedy Mopper:

```
function finalDecisionMopper():
    updateDynamicStrategy() // Cek cat < 40% untuk cari Tower, atau cat > 30 untuk transfer ke kawan

    if isActionReady() then
        // Prioritas 1: Mop Swing (Greedy Area Action)
        bestDir ← null
        maxScore ← -infinity

        for each dir in [North, East, South, West]:
            // ActionPower: Musuh (3), Cat Musuh (2)
            score ← calculateSwingDamage(dir)
            if score > maxScore then
                maxScore ← score
                bestDir ← dir

        if bestDir != null and maxScore > 0 then
            executeMopSwing(bestDir)
            return

        // Prioritas 2: Serang ubin musuh tunggal
        target ← findClosestEnemyTile()
        if target != null then
            executeAttack(target)

    if isMovementReady() then
```

```
    // Greedy Movement: Berdasarkan kuadran dengan cat musuh terbanyak
    targetQuadrant ← findQuadrantWithMostEnemyPaint()
    bestDir ← greedyMoveTo(targetQuadrant)
    move(bestDir)
```

Strategi Greedy Splasher:

```
function finalDecisionSplasher():
  updateDynamicStrategy() // Jika cat < 20%, aktifkan mode Refuel

  if isActionReady() then
    // Prioritas 1: Splash di Ruin
    if findRuinInRange() then
      markTowerPattern()
      executeSplash(ruinLocation)
      return

    // Prioritas 2: Splash Area Damage
    bestTarget ← null
    maxScore ← -infinity

    for each tile in attackableTiles:
      // ActionPower: Hitung total ubin non-ally dalam radius ledakan 2
      score ← calculateSplashScore(tile, radius←2)
      if score > maxScore then
        maxScore ← score
        bestTarget ← tile

    if bestTarget != null and maxScore >= 3 then
      executeAttack(bestTarget)

  if isMovementReady() then
    // Sama dengan Soldier, bergerak mendekati Ruin atau target kuadran
    bestDir ← greedyMoveTo(closestRuin or targetLoc)
    move(bestDir)
```

Strategi Greedy Tower

```
function finalDecisionTower():
  // Pertahanan Otomatis
  if isActionReady() then
    target ← senseClosestEnemy()
    if target != null then executeAttack(target)
```

```
// Greedy Production
if isActionReady() then
   updateDynamicStrategy() // Cek apakah butuh Mopper (banyak kawan low paint)

   unitToBuild ← null
   if round < 80 and not needMopper then
      unitToBuild ← SOLDIER
   else if needMopper then
      unitToBuild ← MOPPER
   else
      unitToBuild ← SPLASHER // Default high-tier

   if canAfford(unitToBuild) then
      bestDir ← findEmptySpace()
      buildRobot(unitToBuild, bestDir)
```

b) Pengujian:

Proses pengujian bot dilakukan melalui terminal dengan mengeksekusi perintah:

./gradlew run -PteamA=alt_bot_2 -PteamB=main_bot-Pmaps=DefaultSmall

./gradlew run -PteamA=alt_bot_2 -PteamB=alt_bot_1-Pmaps=DefaultSmall

Perintah tersebut akan menjalankan simulasi pertandingan antara alt_bot_2 melawan alt_bot_1 dan main_bot pada peta DefaultSmall, di mana main_bot berperan sebagai Team A. Pengujian dengan skema ini memungkinkan kamu untuk melihat apakah main_bot memiliki performa yang konsisten saat menghadapi berbagai tipe lawan yang ditempatkan pada alt_bot.

c) Hasil:

```
[B: #10976@1755] I'm alive
[B: #13726@1765] I'm alive
[B: #10236@1773] I'm alive
[B: #10618@1783] I'm alive
[B: #12564@1791] I'm alive
[B: #11356@1800] I'm alive
[B: #12310@1810] I'm alive
[B: #12733@1818] I'm alive
[B: #13999@1828] I'm alive
[B: #13868@1836] I'm alive
[B: #13394@1846] I'm alive
[B: #13437@1856] I'm alive
[B: #13775@1865] I'm alive
[B: #11690@1875] I'm alive
[B: #11257@1885] I'm alive
[B: #10808@1893] I'm alive
[B: #12630@1903] I'm alive
[B: #11215@1911] I'm alive
[B: #12092@1921] I'm alive
[B: #13027@1931] I'm alive
[B: #12385@1941] I'm alive
[B: #10455@1950] I'm alive
[B: #12163@1958] I'm alive
[B: #13171@1968] I'm alive
[B: #10437@1976] I'm alive
[B: #14046@1986] I'm alive
[B: #12456@1996] I'm alive
[server]                    main_bot (B) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------

BUILD SUCCESSFUL in 3s
8 actionable tasks: 3 executed, 5 up-to-date
```

```
> Task :run
[server] -------------------- Match Starting --------------------
[server] alt_bot_2 vs. alt_bot_1 on DefaultSmall
[server]                    alt_bot_1 (B) wins (round 2000)
[server] Reason: The winning team won on tiebreakers (painted more of the map).
[server] -------------------- Match Finished --------------------

BUILD SUCCESSFUL in 3s
8 actionable tasks: 3 executed, 5 up-to-date
```

Bab 5

Kesimpulan dan Saran

a) Kesimpulan

Berdasarkan implementasi pada Main Bot serta Bot Alternatif, penggunaan algoritma greedy dalam permainan Battlecode 2025 dapat disimpulkan sebagai berikut:

1. Prinsip Pengambilan Keputusan Lokal

Semua bot yang dikembangkan menerapkan prinsip dasar greedy dengan mengambil keputusan terbaik pada setiap giliran (local optimum) berdasarkan informasi yang tersedia di dalam radius sensor (sqrt(20)). Karena keterbatasan jarak pandang dan memori antar robot (independen), strategi greedy menjadi pendekatan yang paling efisien untuk menentukan aksi tanpa memerlukan koordinasi global yang kompleks.

2. Komponen Fungsi Seleksi dan Scoring

Keberhasilan bot sangat ditentukan oleh fungsi seleksi yang diwujudkan dalam sistem penilaian (scoring system). Bot memberikan bobot pada setiap kandidat aksi berdasarkan variabel tertentu:

- Prioritas Unit: Tower menentukan jenis robot yang akan dimunculkan (spawn) berdasarkan kebutuhan sumber daya saat itu atau keberadaan musuh di sekitar.
- Action Power: Robot menghitung nilai utilitas dari setiap petak di sekitarnya dengan mempertimbangkan jumlah cat musuh, keberadaan menara lawan, dan posisi kawan untuk memberikan dukungan (support value).
- Manajemen Jarak: Fungsi pergerakan secara greedy memilih arah yang meminimalkan jarak ke target strategis, seperti reruntuhan (ruins) untuk ekspansi atau menara musuh untuk serangan.

3. Spesialisasi Peran Berbasis Greedy

Setiap tipe robot menerapkan strategi greedy yang berbeda untuk memaksimalkan fungsinya:

- Soldier: Fokus pada ekspansi wilayah dan pembangunan menara di reruntuhan terdekat.
- Splasher: Menggunakan logika area of effect untuk mencari titik serangan yang menghasilkan cakupan warna maksimal dengan biaya cat yang efisien.
- Mopper: Memprioritaskan pembersihan cat musuh dan transfer sumber daya ke unit kawan yang kritis.

4. Efisiensi Komputasi

Algoritma greedy terbukti sangat cocok dengan batasan bytecode (resource komputasi) yang ada di Battlecode. Dengan membuat pilihan instan tanpa perlu melakukan pencarian mendalam

(seperti algoritma minimax atau pencarian jalur yang kompleks di setiap langkah), bot dapat beroperasi dalam batas waktu yang ditentukan oleh engine permainan.

5. Kelemahan dan Strategi Alternatif

Meskipun efisien secara lokal, strategi greedy murni terkadang terjebak dalam kondisi tidak optimal secara global, seperti penumpukan robot di satu titik. Untuk mengatasinya, bot menggunakan variasi parameter dinamis dan prioritas aksi (seperti mendahulukan upgrade menara daripada spawn unit pada kondisi ekonomi tertentu) guna mencapai tujuan akhir yaitu dominasi peta sebesar 70%.

b) Saran:

- Efisiensi Bytecode: Gunakan teknik pemrosesan data yang lebih ringan dan kurangi frekuensi sensing yang berlebihan agar bot tidak melewati batas instruksi per giliran.
- Navigasi dan Kerumunan: Terapkan mekanisme sederhana untuk menghindari penumpukan robot (stacking) di satu titik, misalnya dengan memberikan gaya tolak antar rekan setim.
- Manajemen Sumber Daya: Perketat logika kapan robot harus kembali ke menara sebelum cat benar-benar habis guna menghindari penalti HP dan kecepatan.
- Optimasi Pesan: Manfaatkan fitur komunikasi 4-byte untuk menyebarkan lokasi reruntuhan (ruins) yang ditemukan agar menara bisa segera merespons.
- Adaptasi Fase: Sesuaikan prioritas spawn robot secara otomatis mengikuti perkembangan waktu (Early, Mid, dan Late game) untuk menjaga keseimbangan ekonomi dan militer.

Lampiran

Link Github : https://github.com/fariswirakusuma/Tubes1_KelompokTired

Daftar Pustaka

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Ramadhian Nabil Firdaus Gumay

Faris Wirakusuma

Jonathan Harijadi